

Module 7 — Automation

Lesson 7.3

Scheduled and event-driven agents

Cron, webhooks, queues — anywhere you'd run a script, you can run an agent. The patterns are the same as any other automated job, with a few LLM-specific twists.

Why it matters

Once you have `claude -p` or the SDK working, the deployment shapes are familiar: cron for periodic, webhooks for event-driven, queues for fan-out. The interesting part is the *what* — what jobs justify running an agent vs. a normal script.

Cron / scheduled

Use cases that earn their keep:

- **Daily digest** — summarize what happened in your repos / inboxes / dashboards
- **Stale-PR nag** — find PRs idle >N days and post in the channel
- **Doc drift detection** — compare a doc to current code, flag mismatches
- **Cost-anomaly summary** — pull yesterday's cloud spend, explain the deltas
- **On-call brief** — at handoff time, brief the next on-call with open incidents and what was tried

System cron:

```
# /etc/cron.d/claude-daily-digest
0 9 * * 1-5 ops claude -p "$(cat /opt/jobs/daily-digest-prompt.txt)"
    --dangerously-skip-permissions --output-format json | /opt/jobs/post-to-slack.sh
```

Or in a containerized environment, the same pattern with whatever scheduler you use (Kubernetes CronJob, GitHub Actions schedule, AWS EventBridge).

The Claude Code harness has its own scheduled-execution primitives (look for `CronCreate` / `ScheduleWakeup` in the deferred tools list if you're inside a session). For ad-hoc one-machine scheduling, those work. For production, use real infrastructure.

Event-driven

Webhook handlers are the canonical pattern:

```
incoming event ■■■ web server ■■■ agent task ■■■ store result / take action
```

Common events:

- **GitHub webhook** — issue opened, PR review requested, push to main
- **Linear / Jira webhook** — ticket created, status changed
- **Slack slash command** — `/triage <link>` triggers a Claude analysis
- **Email** — inbound email triggers a draft reply
- **Sentry / monitoring alert** — incident fires, Claude pulls context and posts a starting hypothesis

For one-machine prototyping, a tiny FastAPI app + the SDK:

```
from fastapi import FastAPI, Request
from claude_agent_sdk import query, ClaudeAgentOptions

app = FastAPI()

@app.post("/webhook/issue")
async def on_issue(req: Request):
    payload = await req.json()
```

```
task = f"Triage this issue and suggest labels:\n{payload['issue']['body']}"
result = []
async for event in query(prompt=task, options=ClaudeAgentOptions(max_turns=10)):
    result.append(event)
return {"ok": True}
```

For production, you want a queue between the webhook and the agent — handles retries, rate-limits, and backpressure cleanly.

The queue pattern

```
webhook ■■■ enqueue(task) ■■■ worker ■■■ agent(task) ■■■ result
```

Why a queue:

- Webhooks need to respond fast (within seconds). Agent runs can take minutes.
- Rate limits — you can't run 1000 concurrent agents. The queue paces you.
- Retries — failed runs go back on the queue.
- Visibility — queue depth is your operational health signal.

Any queue works (Redis with Bull/RQ, AWS SQS, GCP Pub/Sub, etc.).

Idempotency

Critical for any production agent. The same event should not double-act if delivered twice (webhooks retry; cron can overlap; manual re-runs happen).

Patterns:

- **Idempotency key per event** — store it; check before acting; refuse duplicates.
- **External-state checks** — before posting a comment, check whether you've already posted one.
- **Dry-run mode** — every agent should have a flag that does everything except the actual side effect, for testing.

Observability essentials

For every scheduled or event-driven agent, you want:

Metric	Why
Invocations per hour	Catch runaway loops
Success rate	Detect regressions early
Average duration	Watch for "the agent is getting confused"
Cost per run + total cost/hour	Surprise bills are bad
Cache hit rate (if SDK-based)	Detect cache drift
Queue depth	Backpressure indicator

A handful of Grafana panels and a Slack alert per metric covers most of it.

When to NOT use an agent

If the task is fully deterministic, write a script. Agents are valuable when:

- The output benefits from natural-language judgment
- Inputs are unstructured (emails, issues, logs)
- The action depends on synthesizing multiple sources
- Rules would be brittle (the LLM's flexibility is the point)

If you can write the rules cleanly, don't pay LLM costs to apply them.

A worked example: stale-PR nag

Daily, find PRs idle >7 days, draft a context-aware nudge per PR, post to the team channel:

```
# nag.sh
set -e
PRS=$(gh pr list --state open --json number,title,url,updatedAt \
      | jq '[.[] | select((now - (.updatedAt | fromdate)) > 604800)]')

if [ "$(echo "$PRS" | jq length)" -eq 0 ]; then
  exit 0
fi

PROMPT="$(cat <<EOF
Draft a friendly Slack message for our team highlighting these stale PRs:
$PRS

For each: one-line reminder, mention any blockers visible from the title, suggest an action.
Tone: helpful, not naggy.
EOF
)"

MSG=$(echo "$PROMPT" | claude -p --dangerously-skip-permissions)
curl -s -X POST "$SLACK_WEBHOOK" -d "{\"text\": $(echo "$MSG" | jq -Rsa .)}"
```

Run from cron daily at 09:00 weekdays. Cost is negligible. Value compounds.

Try it

- 1 Pick a recurring summarization task you currently do manually.
- 2 Write the prompt that does it (test interactively first).
- 3 Wrap it in a shell script that gathers inputs and pipes them into `claude -p`.
- 4 Run it from cron (or GitHub Actions schedule).
- 5 Add logging. Watch for a week. Tune.

Gotchas

- **Overlapping cron runs.** If your job runs every minute and takes 90 seconds, you'll have multiple instances racing. Use a lock file or set the schedule conservatively.
- **Rate limits.** Anthropic enforces per-account rate limits. Concurrent agent runs share them.

- **Cost overruns.** A scheduled job that grew silently from 5 to 500 invocations/day is a Monday morning surprise. Alert on cost rate.
- **Webhook payload validation.** Verify signatures; don't act on unverified inputs.
- **Don't expose webhook endpoints publicly without auth.** Bots will find them and hammer them.

Further reading

- 6.5 Production patterns — observability, retries, kill switches
- examples/06-github-action — GitHub-native scheduled patterns

Next

→ 8.1 Capstone project